Totistack v2.2.13 Security Architecture & Auth/ Audit Control Plane

Status

Version: 2.2.13
Scope: Auth, audit, access control, Cloud Functions, custom claims, signup policy, server actions, Firestore security rules, and initial `sys_admin` bootstrap.

This document defines the final security architecture expected for Totistack v2.2.13. It is written as an implementation and verification reference, not as marketing material.

---

1. Objective

Totistack v2.2.13 must not rely on simple `admin` versus `user` authorization. The framework must support a proper access-control model suitable for production systems and SOC 2-aligned security architecture.

The objective is to make privileged access changes server-authoritative, auditable, versioned, and resistant to client-side privilege escalation.

The system must enforce these principles:

1. The browser never writes privileged role or permission fields.
2. The browser never creates authoritative audit evidence.
3. Role and permission changes go through Cloud Functions.
4. Custom claims are generated by trusted server code only.
5. Signup behavior follows a system policy: public signup, invite-only signup, or disabled signup.
6. The first `sys_admin` is created only by a one-time Admin SDK bootstrap script.
7. Explicit denied permissions override role and direct grants.
8. Stale access tokens are controlled through `accessVersion`.
9. Audit logs are append-only and server-written.
10. Firestore rules enforce the security boundary even if UI guards fail.

---

2. Security Model

Totistack v2.2.13 uses layered access control.

2.1 Layers

Layer	Purpose	Trusted?
UI route guards	Hide pages and block navigation in the frontend	No
Store/service checks	Prevent accidental misuse in application code	No
Server actions	Controlled client interface to backend authority	Partially
Cloud Functions	Authoritative privileged operations	Yes
Firebase custom claims	Fast authorization context in rules and app state	Yes, if issued by Admin SDK
Firestore rules	Final database security boundary	Yes
Audit logs	Evidence trail for privileged actions	Yes, if server-written

2.2 Core rule

Frontend checks improve UX. They are not the source of truth.

The source of truth is:

```
Cloud Functions + Firebase Admin SDK + Firestore Rules + server-written audit logs
```

---

3. Roles and Permissions

3.1 Default roles

Recommended baseline roles:

Role	Purpose
`user`	Standard authenticated user
`manager`	Operational manager with limited administrative access
`admin`	Application administrator
`security-admin`	Can manage roles, permissions, invites, and access policy
`auditor`	Can read audit/security evidence but cannot mutate access
`sysadmin`	Highest privileged break-glass/system owner role

3.2 Permission-first model

Routes, actions, and backend functions should check permissions, not only roles.

Example permissions:

```
auth.users.read
auth.users.update
auth.users.suspend
auth.roles.assign
auth.permissions.grant
auth.permissions.deny
auth.invites.create
audit.logs.read
audit.logs.export
security.policies.update
```

3.3 Explicit deny

Denied permissions must override every grant.

Evaluation order:

```
1. User must be active.
2. User must satisfy MFA/step-up requirement where required.
3. Explicit denied permission blocks access.
4. Direct user permission grants are checked.
5. Role-derived permissions are checked.
6. If none match, access is denied.
```

---

4. Signup Policy

Signup is not hardcoded. It is controlled by security policy.

Policy document:

```
security_policies/auth.default
```

Expected fields:

```
{
  allowPublicSignup: false,
  inviteOnly: true,
  signupDisabled: false,
  defaultSignupRole: 'user',
  requireEmailVerification: true,
  requireMfaForPrivilegedRoles: true,
  policyVersion: 1
}
```

4.1 Supported modes

Mode	Behavior
Public signup	User can register without invite if `allowPublicSignup` is true
Invite-only signup	User must provide a valid invite token
Disabled signup	New signup is blocked

4.2 Required behavior

Public signup may only create a least-privilege account.

The signup payload must not accept:

```
role
roles
permissions
permissionKeys
directPermissionKeys
deniedPermissionKeys
status
mfaRequired
accessVersion
```

These fields must be stripped or rejected server-side.

---

5. Initial sys_admin Bootstrap

The first `sysadmin` must be created through a one-time server-side seed script using Firebase Admin SDK.

It must not be created through:

- public registration;
- invite link;
- frontend seeder;
- Firestore client write;
- browser console;
- normal app UI.

5.1 Seeder command

```
cd functions

SYS_ADMIN_EMAIL="owner@example.com"
SYS_ADMIN_PASSWORD="replace-with-strong-password"
SYS_ADMIN_DISPLAY_NAME="System Administrator"
npm run seed:sys-admin
```

5.2 Seeder responsibilities

The seeder must:

1. Create or update the Firebase Auth user.
2. Create or update the Firestore user profile.
3. Assign the `sysadmin` role.
4. Generate custom claims.
5. Set `accessVersion`.
6. Write an audit event.
7. Mark the account as active.
8. Require MFA if supported by the project.

---

6. Cloud Functions Control Plane

Privileged operations belong in Cloud Functions.

Required functions:

```
completeSignup
createUserBySignup
createUserByInvite
createInvite
assignUserRole
grantUserPermission
denyUserPermission
```

```
suspendUser
restoreUser
revokeUserSessions
syncUserClaims
syncCurrentUserClaims
recordAuditEvent
```

6.1 Function responsibilities

Function	Responsibility
`createUserBySignup`	Server-controlled public signup if policy allows it
`createUserByInvite`	Create user from valid invite token
`completeSignup`	Complete post-auth profile creation under policy
`createInvite`	Create invite token for approved user onboarding
`assignUserRole`	Assign role after hierarchy and permission checks
`grantUserPermission`	Grant direct permission to a user
`denyUserPermission`	Add explicit denied permission
`suspendUser`	Disable application access
`restoreUser`	Restore suspended user access
`revokeUserSessions`	Revoke Firebase refresh tokens and increment access version
`syncUserClaims`	Rebuild claims for a user
`syncCurrentUserClaims`	Refresh caller claims from server-side profile
`recordAuditEvent`	Server-side audit evidence writer

6.2 Privileged function requirements

Every privileged function must:

1. Require authentication.
2. Load the caller profile.
3. Verify caller status is active.
4. Check required permission.
5. Check role hierarchy where applicable.
6. Reject stale or unsafe requests.
7. Write audit evidence.
8. Sync custom claims if access changed.
9. Revoke sessions if access changed materially.

---

7. Server Actions

Client code must call privileged backend operations through server actions.

Expected file:

```
src/features/auth/server-actions/auth.server-actions.js
```

Example usage:

```
const authServerActions = createAuthServerActions({ functions })

await authServerActions.assignUserRole(uid, 'manager', {
  reason: 'Approved operational access change.',
})
```

The root store must inject server actions into the auth access service:

```
createAuthAccessService({
  auth,
  state,
  collectionActions,
  storeApi,
  serverActions: authServerActions,
  config,
})
```

If `functions` is not passed into auth, the implementation is incomplete.

---

8. Custom Claims

Custom claims must be generated by Cloud Functions using Firebase Admin SDK.

Expected claim structure:

```
{
  roles: ['user'],
  permissions: ['auth.profile.read'],
  deniedPermissionKeys: [],
  accessVersion: 1,
  status: 'active',
```

```
  mfaRequired: false,
  mfaEnrolled: false,
  authzVersion: '2.2.13'
}
```

8.1 Claim rules

1. Claims are never written by the browser.
2. Claims are synced after role or permission changes.
3. Claims are synced after suspension/restoration.
4. Claims are synced after invite acceptance.
5. Claims are synced after access policy changes where required.
6. `accessVersion` must increment after material access changes.
7. Token refresh must be required after claim changes.

---

9. Firestore Rules

Firestore rules must enforce the database boundary.

9.1 Client must not write privileged fields

Blocked fields:

```
role
roles
permissions
permissionKeys
directPermissionKeys
deniedPermissionKeys
status
mfaRequired
accessVersion
securityProfile
claims
```

9.2 Client must not write security collections

The following collections must be server-only or restricted to privileged server-validated reads:

```
roles
permissions
audit_logs
security_policies
```

```
user_invites
sessions
access_reviews
security_events
```

9.3 Own-profile update limit

A user may update only safe profile fields, such as:

```
displayName
photoURL
phoneNumber
preferences
updatedAt
```

No role, permission, status, or security fields may be changed through a normal client profile update.

---

10. Audit Architecture

Audit logs must be written by backend authority, not by normal frontend code.

10.1 Required audit fields

```
{
  actorUid: 'uid',
  actorEmail: 'user@example.com',
  actorRoles: ['security-admin'],
  action: 'auth.roles.assign',
  targetType: 'user',
  targetId: 'uid',
  decision: 'allowed',
  reason: 'Approved access change.',
  controlIds: ['CC6.1', 'CC6.2'],
  correlationId: 'uuid',
  evidenceHash: 'sha256...',
  previousHash: 'sha256...',
  createdAt: serverTimestamp(),
  source: 'cloud-functions',
  authzVersion: '2.2.13'
}
```

10.2 Secret sanitization

Audit payloads must sanitize:

```
password
token
secret
cookie
apiKey
authorization
session
refreshToken
idToken
```

Audit logs must never contain raw credentials, tokens, cookies, or secrets.

10.3 Immutability expectation

Firestore audit documents should be append-only. Client update/delete must be denied.

For stronger evidence retention, export audit logs to long-term storage or a dedicated log sink.

---

11. Session Revocation

When access changes materially, the system must revoke existing refresh tokens.

Examples requiring revocation:

- role changed;
- privileged permission granted;
- explicit denial added;
- user suspended;
- user restored;
- password reset forced;
- suspicious activity detected;
- MFA disabled or reset.

Expected backend action:

```
await adminAuth.revokeRefreshTokens(uid)
```

Expected profile update:

```
{
  accessVersion: increment(1),
  sessionsRevokedAt: serverTimestamp(),
  sessionsRevokedBy: callerUid,
  sessionsRevokedReason: reason
}
```

Firestore rules and app guards must compare token `accessVersion` with profile `accessVersion` where possible.

---

12. MFA and Step-Up Authentication

v2.2.13 must support the architecture for MFA enforcement, even if a project implements the full MFA UI later.

Required metadata:

```
mfaRequired
mfaEnrolled
mfaVerifiedAt
lastReauthAt
```

Privileged actions should require one of:

- verified MFA session;
- recent reauthentication;
- approved break-glass path.

MFA events must be audited:

```
mfa.enrolled
mfa.disabled
mfa.reset_requested
mfa.reset_approved
mfa.step_up_passed
mfa.step_up_failed
```

---

13. Release Integrity Requirements

A final v2.2.13 release must have consistent version references.

Required:

```
root package.json version = 2.2.13
CLI version = 2.2.13
auth feature manifest version = 2.2.13
audit feature manifest version = 2.2.13
server-actions feature manifest version = 2.2.13
security docs version = 2.2.13
```

A release is not final if generated templates still reference older versions such as `2.0.0` or `2.2.0`.

---

14. Implementation Checklist

Before calling v2.2.13 final, verify:

```
[ ] Root package version is 2.2.13.
[ ] CLI reports 2.2.13.
[ ] Auth manifest reports 2.2.13.
[ ] Audit manifest reports 2.2.13.
[ ] Server-actions manifest reports 2.2.13.
[ ] Generated app store imports Firebase Functions.
[ ] Generated app store creates auth server actions.
[ ] Auth access service receives serverActions.
[ ] Public signup calls backend policy flow.
[ ] Invite signup reserves invite before creating user.
[ ] Browser cannot write roles or permissions.
[ ] Browser cannot write audit logs.
[ ] Browser cannot write security policies.
[ ] Role assignment goes through Cloud Function.
[ ] Permission grant/deny goes through Cloud Function.
[ ] Claims sync runs after access changes.
[ ] Session revocation runs after material access changes.
[ ] sys_admin seeder uses Admin SDK only.
[ ] Firestore rules block direct privileged writes.
[ ] Audit payloads sanitize secrets.
[ ] Tests cover privilege escalation attempts.
```

---

15. Required Tests

Minimum test coverage:

15.1 Unit tests

```
[ ] denied permission overrides allowed permission.
[ ] inactive user is denied.
[ ] suspended user is denied.
[ ] admin cannot assign sysadmin.
[ ] security-admin cannot assign sysadmin unless explicitly allowed.
[ ] sysadmin can assign lower roles.
[ ] public signup strips privileged fields.
[ ] invite signup strips privileged fields.
[ ] claims payload contains only safe claim data.
[ ] audit sanitizer removes secrets.
```

15.2 Firestore emulator tests

```
[ ] user cannot update own role.
[ ] user cannot update own permissions.
[ ] user cannot update own status.
[ ] user cannot create audit log.
[ ] user cannot delete audit log.
[ ] admin claim cannot bypass forbidden field update.
[ ] stale accessVersion blocks protected access.
[ ] auditor can read audit logs if policy allows.
[ ] normal user cannot read audit logs.
```

15.3 Function tests

```
[ ] createUserBySignup respects public signup policy.
[ ] createUserBySignup rejects signup when disabled.
[ ] createUserByInvite rejects invalid token.
[ ] createUserByInvite prevents token reuse.
[ ] assignUserRole writes audit event.
[ ] assignUserRole syncs claims.
[ ] assignUserRole revokes sessions.
[ ] suspendUser disables access and revokes sessions.
[ ] restoreUser syncs claims and writes audit event.
```

---

16. SOC 2 Alignment

This architecture supports SOC 2-aligned controls but does not make a company SOC 2 compliant by itself.

SOC 2 still requires operational controls, evidence, policies, monitoring, vendor management, incident response, access reviews, change management, and auditor examination.

16.1 Relevant control areas

Area	System Support
Access control	RBAC, permissions, explicit deny, custom claims
Least privilege	default user role, invite policy, role hierarchy
Logical access review	access review collections and audit events
Change management	audit logs for policy and permission changes
Security monitoring	audit/security events and notification hooks
Incident response	session revocation, suspension, audit evidence
Data protection	Firestore rules and privileged field blocking
Evidence retention	append-only audit logs and export-ready schema

---

17. Non-Negotiable Architectural Commitments

These are the final commitments for Totistack v2.2.13:

```
1. No client-side privileged writes.
2. No browser-created sysadmin.
3. No frontend role assignment.
4. No public signup role selection.
5. No audit evidence from untrusted clients.
6. No direct security policy mutation from the browser.
7. No role-only authorization where permission checks are required.
8. No stale access after material permission change.
9. No secret leakage into audit logs.
10. No final release with version mismatch.
```

---

18. Remaining Future Work

These items may remain outside v2.2.13 if explicitly accepted, but they should not be forgotten:

1. Full Firebase MFA enrollment and recovery UI.
2. Firestore emulator test suite in the framework repository.
3. External WORM-style audit log retention.

4. Automated access review workflow.
5. Security alert integration with notifications.
6. Break-glass approval workflow with expiry.
7. Admin dashboard for audit evidence review.
8. SIEM/export adapter.

---

19. Final Position

The correct architecture is:

```
Public app / Admin UI
        ↓
Server Actions
        ↓
Cloud Functions
        ↓
Firebase Admin SDK
        ↓
Custom Claims + Firestore Writes + Audit Logs
        ↓
Firestore Rules
```

Any implementation that allows the browser to directly write roles, permissions, audit evidence, security policies, or sysadmin users is not production-grade and should not be released as Totistack v2.2.13.